Augmenting LLM Code Translation with Compiler Analysis for C to Triton Kernel Generation

Xiao Qin
University of Leeds
Leeds, United Kingdom
ppnh4382@leeds.ac.uk

Chunwei Xia
University of Leeds
Leeds, United Kingdom

Zheng Wang
University of Leeds
Leeds, United Kingdom

ABSTRACT

Emerging programming models like Triton enable developers to better exploit modern accelerators, but translating legacy code to Triton remains challenging. While Large Language Models (LLMs) show promise for code translation, they often generate incorrect or suboptimal implementations due to a lack of precise parallelization reasoning. We present TritonPilot, a compiler-assisted LLM framework for translating C loop nests to Triton kernels. TritonPilot extracts parallelization properties using compiler-based analysis and encodes them as structured analysis facts in the LLM prompt, enabling correct and performance-aware parallel code synthesis. A profiling feedback loop further improves performance by feeding GPU hardware utilization metrics back to the LLM for iterative optimization. Across PolyBench/C and TSVC kernels, TritonPilot achieves near-perfect correctness (100% and 99%), improving over LLM-only baselines. On TSVC at realistic data sizes, the generated Triton kernels achieve 1.02× median performance vs. OpenMP GPU offloading compiled by NVIDIA's production compiler on the same GPU. Profiling feedback improves 15 of 30 PolyBench kernels, with gains up to 47× on individual kernels. We further demonstrate generalization on Rodinia and ECP proxy applications. These results show that combining compiler analysis with hardware profiling feedback enables both correct and high-performance LLM-based parallel code generation.

CCS CONCEPTS

• **Software and its engineering** → **Compilers**; • **Computing methodologies** → *Parallel computing methodologies*.

KEYWORDS

LLM code generation, GPU kernels, Triton, polyhedral analysis

ACM Reference Format:
Xiao Qin, Chunwei Xia, and Zheng Wang. 2026. Augmenting LLM Code Translation with Compiler Analysis for C to Triton Kernel Generation. In *Proceedings of Workshop on AI for High Performance Compiler Construction (AI4HPCC '26).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

AI4HPCC '26, July 6–9, 2026, Belfast, Northern Ireland, UK
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM. . . $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Accelerators such as GPUs are now essential for high-performance scientific computing, yet much existing software is still written in legacy C code that is not designed for parallel execution on modern accelerators. Translating these legacy programs to accelerator-friendly implementations is challenging, as it requires identifying parallelism, restructuring computation, and managing synchronization, tasks that require expertise in programming models and hardware.

Emerging programming models such as Triton [1] simplify GPU programming by providing higher-level abstractions for writing efficient kernels. However, mapping legacy C code to Triton still requires careful reasoning about loop-level parallelism, data dependencies, and execution order. Developers must determine which loop dimensions can be safely parallelized, how memory accesses should be organized, and when synchronization is required - decisions that are subtle and error-prone in practice.

Large Language Models (LLMs) have recently shown promise in code translation, including generating GPU kernels from high-level descriptions [2, 3]. However, LLMs lack precise reasoning about program semantics, especially data dependencies. As a result, they often produce incorrect or inefficient code, including unsafe parallelization that leads to data races, suboptimal kernel structures, and missing synchronization.

In this paper, we present TritonPilot, a system that augments LLM-based translation with compiler analysis. TritonPilot extracts parallelization properties from C loop nests using polyhedral analysis (PET) [4] and LLVM dependence analysis [5], and encodes them as structured guidance in the LLM prompt. This allows the model to generate correct and performance-aware Triton code without requiring it to perform dependence analysis itself.

We evaluate TritonPilot on a diverse set of benchmarks and application kernels on an NVIDIA RTX 3090 GPU. Across PolyBench/C and TSVC loop kernels, TritonPilot achieves near-perfect correctness (100% and 99%), significantly improving over LLM-only baselines. At realistic data sizes, the generated Triton kernels match the performance of OpenMP GPU offloading compiled by NVIDIA's `nvc` compiler (1.02× median on the same GPU). We further demonstrate generalization on kernels from Rodinia and ECP proxy applications. This paper makes the following contributions:

- A compiler-assisted LLM framework for translating legacy C loop nests to Triton, with a unified analysis module that encodes parallelization constraints as structured facts rather than pattern-specific templates.

- A profiling-guided optimization loop that feeds GPU hardware utilization metrics (from NVIDIA Nsight Compute) back to the LLM, enabling iterative performance improvement while preserving correctness.
- Empirical results showing that compiler analysis ensures correctness (100% on PolyBench/C) and that profiling feedback recovers the performance of hand-tuned prompt templates while maintaining generality.

2 OUR APPROACH

TritonPilot is implemented in approximately 14K lines of Python, using PET/isl for polyhedral analysis and LLVM 17 for fallback dependence analysis. Figure 1 depicts the workflow of TritonPilot using `gemm` as a running example. Given the source program (i.e., a C source file in this work), TritonPilot automatically identifies optimisation candidates and translates them into the target language (i.e., Triton kernels in this work). The TritonPilot pipeline consists of six stages: loop nest extraction, static analysis, unified prompt construction, LLM-based generation, iterative refinement, and profiling-guided optimization.

2.1 Loop Nest Extraction

The current implementation of TritonPilot targets loop-level code translation. It leverages PET's Static Control Part (SCoP) detection [4] to identify affine loop nests suitable for transformation. It scans each function and selects regions where loop bounds and memory accesses are affine in loop iterators and parameters, requiring no manual annotations. For example, for `gemm` in Figure 1, PET extracts a SCoP covering the full loop nest and produces a polyhedral representation including iteration domains, access relations, and loop schedules. This representation provides a precise and structured basis for subsequent dependence analysis.

2.2 Static Analysis

TritonPilot performs dependence-driven analysis to derive constraints that govern correct parallelization and code generation. The analysis is organized as a set of independent modules that each examine one aspect of the kernel, producing results in a unified structured representation (a JSON schema with fixed fields regardless of what patterns the kernel exhibits). This design ensures that adding a new analysis module does not require modifying the prompt construction logic, improving extensibility and maintainability. The current implementation includes the following analyses:

Parallelization Analysis. For each loop dimension, TritonPilot determines whether it can be safely parallelized. This involves checking that no cross-iteration data dependences exist for that dimension. In `gemm`, both `i` and `j` are parallelizable since updates to `C[i][j]` do not introduce cross-iteration conflicts. Dimensions that carry dependences are marked as invalid with an explanation (e.g., "Dependencies carried by loop `k`").

WAR Dependence Analysis. Write-after-read (WAR) dependences are computed using exact polyhedral analysis to determine whether in-place updates are safe or require array cloning. When loop-level scoping information is available (via LLVM direction vectors), the analysis reports which loops carry the WAR, enabling finer-grained decisions. For `gemm`, no WAR dependences are present.

Memory Access Pattern Detection. TritonPilot identifies high-level memory access patterns that affect kernel structure: *(i)* **Stencil patterns**: neighboring-element reads of the same array, requiring synchronization barriers (`tl.debug_barrier()`) between phases within a single CTA. *(ii)* **Triangular access**: loop bounds that depend on outer loop variables (e.g., `j < i`), suggesting one-program-per-row grid strategies. *(iii)* **Cross-phase dependencies**: multi-phase computations where one phase writes what another phase reads, requiring either separate kernel launches or explicit barriers.

Reduction and Scalar Expansion. Reduction patterns (e.g., sum, max) are detected and mapped to Triton primitives (`tl.sum()`, `tl.dot()`). Scalars that accumulate across parallel iterations are identified for expansion to per-thread registers, preventing shared-state race conditions.

LLVM Fallback. When polyhedral analysis is not applicable (e.g., due to non-affine accesses), TritonPilot falls back to LLVM's `DependenceAnalysis` to conservatively infer dependence-carrying loops. Direction vectors from LLVM are parsed and normalized to augment or replace polyhedral results.

2.3 Unified Prompt Construction

All analysis modules produce results in a common schema. A single prompt construction function (`format_analysis_for_prompt`) renders the analysis into structured text that is pattern-agnostic: it emits whatever constraints the analysis found, without pattern-specific templates. This design principle—*tell the LLM what is true about the kernel, rather than prescribing what to do*—improves scalability and reduces maintenance burden. The prompt includes:

- The original C source code and loop body to translate.
- Analysis results: valid/invalid parallelization dimensions with reasons, WAR dependence information, detected memory access patterns, and reduction types.
- The required Triton function signature and coding rules (e.g., vectorized access patterns, `tl.constexpr` usage).

The LLM receives the *facts* ("dimension `j` is safe to parallelize; dimension `t` has cross-phase dependencies") and decides the implementation strategy, rather than following a prescriptive code template.

2.4 Generation and Iterative Refinement

Given the constructed prompt, TritonPilot invokes the LLM (Claude Sonnet 4 [7] in this work) to generate the target code. The generated kernel is validated against the

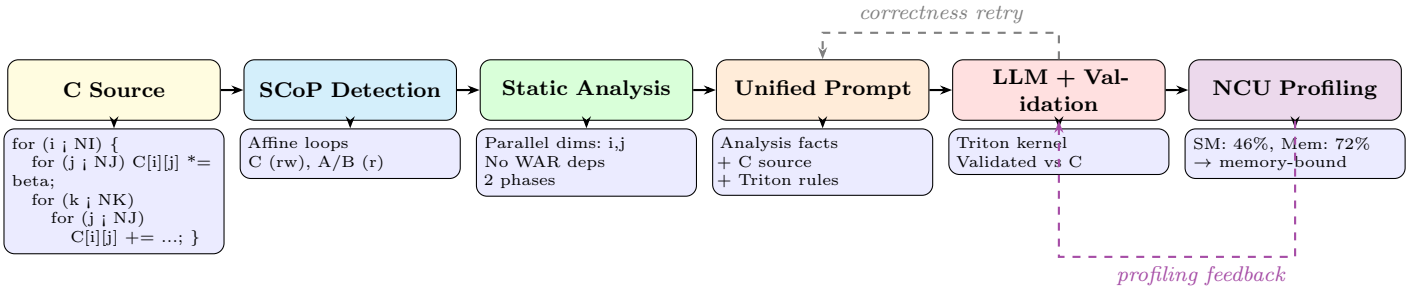

Figure 1: The TritonPilot workflow. Dashed arrows show iterative loops: the gray loop retries on correctness failures; the violet loop feeds NCU profiling metrics back to the LLM for performance optimization.

source program (compiled by LLVM in our case) using identical test inputs and FP32 tolerance checks. On validation failure, TritonPilot classifies the error into one of four categories: *(i)* compilation or runtime errors (syntax, missing imports, Triton JIT failures); *(ii)* numerical errors (incorrect results beyond tolerance); *(iii)* missing synchronization barriers (detected via pre-validation for known stencil patterns); *(iv)* low performance (<0.1× speedup, indicating insufficient parallelism). Each category triggers a targeted retry prompt with error-specific feedback. This process repeats for up to 10 attempts, with a context reset after the fifth attempt to mitigate error accumulation.

2.5 Profiling-Guided Optimization

After a kernel passes correctness validation and benchmarking, TritonPilot optionally enters a profiling-guided optimization loop to improve performance. This stage uses NVIDIA Nsight Compute (NCU) to collect hardware utilization metrics from the generated kernel, then feeds them back to the LLM as optimization guidance.

Profiling. NCU profiles the kernel and reports: SM (compute) throughput as a percentage of peak, memory throughput as a percentage of peak, warp occupancy, L1/L2 cache hit rates, and kernel duration. From these metrics, TritonPilot classifies the bottleneck as *compute-bound* (high SM utilization), *memory-bound* (high memory throughput), or *latency-bound* (both low, indicating insufficient parallelism or launch overhead).

Feedback Loop. The bottleneck classification, raw metrics, and the current kernel code are included in a new LLM prompt requesting an optimized version. The LLM-generated optimization is then re-validated for correctness and re-benchmarked. If the new version is both correct and faster, it replaces the current best; otherwise, the previous version is retained. This loop runs for up to N iterations (3 in our experiments). To avoid redundant profiling, NCU results are cached and reused when the implementation does not change between iterations.

Bottleneck-Specific Guidance. The optimization prompt includes targeted advice based on the bottleneck type: memory-bound kernels are guided toward shared memory tiling and coalesced access; compute-bound kernels toward `tl.dot()` and vectorization; latency-bound kernels toward increased grid size and reduced sequential work. This structured feedback enables the LLM to make informed optimization decisions rather than guessing.

3 EVALUATION SETUP

Platforms. All experiments are conducted on a GPU workstation equipped with an NVIDIA RTX 3090 GPU (24 GB memory, 936 GB/s memory bandwidth) and a dual-socket Intel Xeon Gold 5218R CPU (80 logical cores, ∼140 GB/s memory bandwidth). We use PyTorch 2.5.1, Triton 3.1.0, CUDA 12.4, and GCC 11.4.0. We use Claude Sonnet 4 [7] as the underlying LLM for code translation.

Benchmarks. We evaluate on three benchmark suites: (1) *PolyBench/C* [6]: 30 numerical kernels (stencils, linear algebra, data mining) at default sizes (N=60–120) and 8× scale (N=480–960); (2) *TSVC* [15]: 151 loop kernels covering linear traversals, reductions, indirect addressing, and loop-carried dependences, benchmarked at 32 MB and 3200 MB per array (N=8M and 800M); (3) *Application kernels*: 8 kernels from Rodinia (SRAD, lud, gaussian, lavaMD, hotspot, pathfinder) and ECP proxy applications (miniMD LJ force, miniFE SpMV).

Baselines. The primary comparison is against *OMP GPU*—OpenMP GPU offloading compiled by NVIDIA `nvc` with managed memory, running on the *same* GPU as Triton. This is the strongest baseline: it isolates the quality of LLM-generated Triton code from hardware differences, directly comparing two GPU programming models on identical hardware. We additionally report results against *OMP CPU* (80-thread OpenMP on CPU) to quantify the GPU's architectural advantage, and *ST* (single-threaded GCC `-O3`) as a conventional reference point.

Evaluation methodology. All speedups measure kernel-only execution time with data pre-staged on the GPU, averaged over 10–50 iterations after warmup. For the correctness

Table 1: Polybench/C results. WA = with analysis, NA = no analysis.

Metric	WA	NA
Pass rate	30/30 (100%)	28/30 (93%)
Median speedup	2.15×	1.06×
Mean speedup	3.44×	1.52×
Kernels > 1×	20/30	14/28
Head-to-head (28 common):		
WA wins (>10% faster)	12	
NA wins (>10% faster)	6 (3 race conditions)	
Ties (within 10%)	10	

ablation, we compare two LLM configurations: *with analysis* (WA), where the prompt includes compiler-derived guidance, and *no analysis* (NA), where the LLM receives only the C source code. This isolates the contribution of static analysis to generation quality.

4 EXPERIMENTAL RESULTS

4.1 Polybench/C (30 Kernels)

Table 1 summarizes results at standard (1×) problem sizes. All 30 kernels pass with analysis guidance; without it, two stencil kernels fail due to parallelizing dimensions with carried dependences. The 2.15× median speedup is limited by Polybench's small defaults ($N = 60$–120), where GPU launch overhead dominates. Compute-bound kernels show strong speedups (`heat_3d`: 12.36×, `deriche`: 7.44×, `covariance`: 7.40×).

In the head-to-head comparison (Table 1), 6 of 28 common kernels appear faster without analysis guidance. However, 3 of these are *false wins*: the unguided LLM parallelizes loop dimensions that carry data dependencies, producing code that runs faster but computes incorrect results. For example, in `lu`, the NA code launches one GPU thread per row (`grid=(N,)`), but row i depends on the result of row k ($k < i$). Without sequential ordering, threads race to read unfinished rows, causing output differences up to 10^{33} across repeated runs. With analysis guidance, TritonPilot correctly identifies this dependence and enforces sequential row processing, sacrificing parallelism for correctness. The remaining 3 NA wins are within measurement noise and reflect nondeterministic LLM code quality rather than a systematic advantage of unguided generation.

Scaling. At 8× sizes (Table 2), median speedup rises to 30.73× over single-threaded C. Against 80-thread OpenMP, the GPU still achieves 8.57× median, confirming genuine advantage beyond CPU parallelism. Five sequential kernels time out during benchmark (`grid=(1,)`).

Guidance Impact. Table 3 shows the impact of individual guidance categories, each triggered by analysis properties (e.g., cross-phase dependences, triangular patterns). The largest gains come from multi-phase decomposition (correlation: 13×) and triangular grid (lud: 94×).

Table 2: Polybench scaling: 1× to 8× problem sizes (N: 60→480, matrices: 120→960). "ST" = single-threaded C; "OMP" = 80-thread OpenMP.

Metric	1×	8× (ST)	8× (OMP)
Pass rate	30/30	30/30	30/30
Benchmarked	30	25*	25*
Median speedup	2.15×	30.73×	8.57×
Kernels > 1×	20/30	21/25	20/25

*5 sequential kernels timeout during benchmark.

Table 3: Impact of analysis-triggered guidance categories. "Before" and "After" are speedups at 1× sizes except where noted.

Guidance	Kernel	Before	After
Multi-phase split	correlation	0.19×	2.50×
WAR sequential	nussinov	FAIL	PASS
Triangular grid	lud†	0.14×	13.18×
Fused reductions	bicg	1.04×	3.51×
Stencil barriers	jacobi_1d	0.13×	1.78×
IIR phase fusion	deriche	0.28×	7.19×
Tridiag. sweep	adi	0.24×	2.27×

†Rodinia benchmark.

Table 4: Effect of profiling feedback on PolyBench/C (1× sizes). "Unified" uses the general-purpose analysis module; "+ Prof" adds 3 NCU profiling-feedback iterations.

Metric	Legacy	Unified	Unified+Prof
Pass rate	30/30	30/30	30/30
Median speedup	1.77×	1.40×	1.90×
Mean speedup	2.37×	1.90×	2.36×
Kernels > 1×	19/30	16/30	19/29
Profiling-improved	—	—	15/30

Profiling-Guided Optimization. Table **??** compares three prompt strategies: the original pattern-specific prompt ("Legacy", with hand-crafted templates per dependence pattern), the unified analysis module ("Unified", a general-purpose renderer with no pattern-specific logic), and unified analysis augmented with NCU profiling feedback ("Unified+Prof").

The unified module alone achieves slightly lower performance (1.40× vs. 1.77× median), which is expected: it provides analysis *facts* rather than prescriptive code templates. Adding profiling feedback closes the gap entirely (1.90× median, 2.36× mean), matching the legacy approach while using 10× fewer lines of prompt-construction logic (80 vs. 870).

Profiling feedback is most effective on latency-bound kernels where the initial implementation has insufficient parallelism: `deriche` improves from 0.12× to 5.69×, `gemver` from 0.15× to 1.98×, `bicg` from 0.40× to 1.90×, and `3mm` from 2.82× to 5.17×. In each case, the NCU metrics identified the

Table 5: TSVC results at three data sizes. "ST" = single-threaded C (`-O2`), "OMP" = OpenMP C (`-O2 -fopenmp`, 80 threads). Pass rate: 150/151 (99.3%).

	32 MB	3200 MB
Per-array size	32 MB	3.2 GB
Parallelizable*	27/150	15/150
Tri/ST median	67.1×	73.2×
Tri/OMP CPU median	10.7×	14.2×
Tri/OMP GPU median	1.02×	—†

*Of 150 passing kernels, only those with parallelizable dimensions are benchmarked; serial and 2D kernels are excluded because they exceed GPU memory at large sizes.
†`nvc -mp=gpu` does not support per-array sizes >32 MB.

Table 6: Application kernels: Triton (WA) vs. OMP GPU offloading (`nvc`) on the same GPU, and WA vs. NA ablation (vs. ST). "—" = failed.

Source	Kernel	Tri/OMP GPU	WA/ST	NA/ST
Rodinia	SRAD	24.9×	24.9×	3.5×
Rodinia	hotspot	4.3×	1.1×	1.2×
Rodinia	gaussian	249×	3.4×	3.6×
miniMD	LJ force	6.8×	25.8×	23.0×
miniFE	SpMV	3.1×	9.0×	7.3×
Rodinia	lud	—[a]	13.2×	15.5×
Rodinia	lavaMD	—[a]	1.5×	0.6×
Rodinia	pathfinder	—[a]	0.1×	—[b]

[a]OMP GPU offloading (`nvc`) fails to compile these kernels.
[b]NA generates incorrect code (fails correctness check).

bottleneck (low SM and memory utilization) and the LLM restructured the kernel to use more parallel resources.

Correctness is preserved across all configurations: the compiler analysis ensures safe parallelization regardless of prompt style, and the profiling loop re-validates each optimized kernel before accepting it. Of the 15 kernels where profiling attempted optimization, all retained correctness; optimization attempts that broke correctness were automatically rejected.

Limitations. Ten kernels achieve <1× at standard sizes: five are inherently sequential (loop-carried dependences limit parallelism), and two (`jacobi_2d`, `fdtd_2d`) suffer from kernel launch overhead at small N—at 8× sizes they reach 36.82× and 15.43×. During profiling-guided optimization, the LLM frequently breaks correctness when attempting performance improvements (the rejected attempts outnumber successful ones), confirming that single-shot optimization is insufficient and motivating integration with search-based or RL-based methods in future work.

4.2 TSVC (151 Kernels)

We apply TritonPilot to 151 TSVC [15] kernels covering linear traversals, reductions, indirect addressing, and loop-carried dependences. Table 4 shows results at realistic data sizes. TritonPilot passes 150/151 (99.3%), 120 on first attempt. At 32 MB, LLM-generated Triton achieves 67× (ST) and 10.7× (80-thread OMP CPU). Crucially, Triton achieves **1.02× vs. OpenMP GPU offloading** (`nvc -mp=gpu`) on the *same* GPU, showing the LLM produces code matching a production compiler.

4.3 Application Kernels

To evaluate generalization, we apply TritonPilot to 6 Rodinia kernels [13] and 2 ECP proxy kernels [14] (Table 5). All 8 pass correctness with analysis guidance (6 on first attempt).

Triton vs. OMP GPU (same hardware). On 5 kernels with OMP GPU baselines, LLM-generated Triton is 3–249× faster than `nvc`'s OpenMP offloading. The advantage comes from Triton's block-level model: the LLM fuses phases, uses shared memory tiling, and avoids per-timestep target-region launch overhead that penalizes `nvc` on iterative kernels (gaussian: 511 timesteps × 3 launches = 1533 offloads). SRAD's multi-phase stencil benefits similarly (24.9×).

WA vs. NA (analysis impact). Analysis guidance is most impactful on multi-phase kernels: SRAD improves from 3.5× (NA) to 24.9× (WA) via phase decomposition; lavaMD improves from 0.56× to 1.47× as analysis identifies the parallelizable dimension (NA fails on 4/5 attempts).

5 RELATED WORK

LLM Code Generation for GPUs. Chen et al. [2] evaluate ChatGPT on CUDA kernel generation, finding frequent correctness issues including data races and incorrect memory management. KernelBench [3] benchmarks LLM-generated Triton/CUDA on 250 tasks, reporting 20–40% functional correctness without guidance. Our work differs fundamentally: rather than relying on the LLM's implicit knowledge of parallelism, we provide explicit compiler-derived constraints, achieving 100% correctness on Polybench/C and 99% on TSVC.

LLM/Agent-Based Kernel Optimization. Recent works improve kernel generation through more sophisticated methods. *RL fine-tuning* approaches train LLMs with execution feedback: Dr. Kernel [**?**] uses profiling-based rewards to avoid reward hacking, achieving 31.6% of kernels with ≥1.2× speedup on KernelBench (beating Claude Sonnet 4.5 and GPT-5); Tehrani et al. [**?**] RL-fine-tune GPT-5, improving single-attempt correctness from 43.7% to 77.0%; DICE [**?**] applies RL to *diffusion* (non-autoregressive) LLMs, enabling holistic structural planning. *Search-based* methods keep the LLM frozen: AVO [**?**] replaces evolutionary operators with LLM agents, discovering attention kernels that outperform FlashAttention-4 by 10.5%; K-Search [**?**] decouples high-level strategy planning from code instantiation via a co-evolving world model, achieving up to 14.3× on MoE kernels; OptiML [**?**] uses MCTS with Nsight Compute profiler feedback. KernelBlaster [**?**] builds a persistent cross-task CUDA knowledge base for continual learning without fine-tuning. For non-GPU targets, AscendKernelGen [**?**] and AscendCraft [**?**]

address Huawei Ascend NPU kernels through domain-specific datasets and intermediate DSLs. These works focus on *how* to generate or optimize kernels (via RL, search, or memory); our work is orthogonal, addressing *what* parallelism is safe through compiler analysis. The two directions are complementary: compiler-derived constraints could serve as reward signals or world-model priors in these frameworks.

Polyhedral Compilation and Auto-Scheduling. Polyhedral frameworks such as Pluto [11] and PPCG [12] automatically generate parallel code from affine loop nests. PPCG targets CUDA directly but is limited to affine programs and produces low-level code that lacks Triton's block-level optimizations (automatic memory coalescing, shared memory tiling). TVM [9] and Halide [10] use auto-scheduling with learned cost models for tensor operations. TritonPilot is complementary: rather than replacing the compiler, it uses polyhedral analysis to extract *what* parallelism is safe, then delegates the *how* to the LLM, which can handle non-affine constructs, irregular access patterns, and high-level Triton abstractions that polyhedral compilers cannot target directly.

6 CONCLUSION

We have presented TritonPilot, a system that combines compiler-based analysis with LLM-based kernel generation for accelerators. TritonPilot encodes parallelization constraints, dependence information, and memory access patterns as structured analysis facts in the LLM prompt, enabling correct and efficient Triton kernel generation from C programs.

TritonPilot achieves near-perfect correctness (30/30 PolyBench/C, 150/151 TSVC) and generates Triton code that matches production compiler quality: on TSVC at 32 MB, LLM-generated Triton achieves 1.02× median vs. OpenMP GPU offloading (`nvc`) on the same GPU. It generalizes to 8 application kernels from Rodinia and ECP proxy apps. Ablation results show that analysis-guided prompting is essential for correctness, preventing failures and exposing latent race conditions in unguided generation.

Our unified analysis module replaces pattern-specific prompt templates with a general-purpose renderer, reducing prompt logic by 10× while maintaining correctness. The profiling feedback loop, which feeds NCU hardware metrics back to the LLM, improves 15 of 30 PolyBench kernels and recovers the performance of the pattern-specific approach (1.90× vs. 1.77× median). These results suggest that compiler analysis and hardware profiling provide complementary signals—correctness constraints and performance guidance—that could serve as reward functions or world-model priors in RL-based or search-based kernel optimization frameworks.

REFERENCES

[1] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proc. MAPL*, 2019.

[2] Y. Chen, S. Huang, D. Zheng, et al. ChatGPT for CUDA programming: Evaluating LLM-generated GPU code. *arXiv preprint arXiv:2404.12000*, 2024.

[3] A. Ouyang, S. Zheng, et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2404.12489*, 2024.

[4] S. Verdoolaege and T. Grosser. Polyhedral extraction tool. In *Proc. IMPACT*, 2012.

[5] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proc. CGO*, 2004.

[6] L.-N. Pouchet. Polybench/C: The polyhedral benchmark suite, version 4.2.1. https://web.cs.ucla.edu/~pouchet/software/polybench/, 2016.

[7] Anthropic. Claude Sonnet 4 model card. https://www.anthropic.com/claude, 2025.

[8] S. Verdoolaege. isl: An integer set library for the polyhedral model. In *Proc. ICMS*, 2010.

[9] T. Chen, T. Moreau, Z. Jiang, et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proc. OSDI*, 2018.

[10] J. Ragan-Kelley, C. Barnes, A. Adams, et al. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proc. PLDI*, 2013.

[11] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proc. PLDI*, 2008.

[12] S. Verdoolaege, J. Carlos Juega, A. Cohen, et al. Polyhedral parallel code generation for CUDA. *ACM TACO*, 9(4), 2013.

[13] S. Che, M. Boyer, J. Meng, et al. Rodinia: A benchmark suite for heterogeneous computing. In *Proc. IISWC*, 2009.

[14] M. A. Heroux, D. W. Doerfler, P. S. Crozier, et al. Improving performance via mini-applications. Sandia National Laboratories Technical Report SAND2009-5574, 2009.

[15] D. Callahan, J. Dongarra, and D. Levine. Vectorizing compilers: A test suite and results. In *Proc. Supercomputing*, 1988.

[16] W. Liu, J. Xu, Y. Li, et al. Dr. Kernel: Reinforcement learning done right for Triton kernel generations. *arXiv preprint arXiv:2602.05885*, 2026.

[17] A. Tehrani, Y. Emara, E. Wissam, et al. Fine-tuning GPT-5 for GPU kernel generation. *arXiv preprint arXiv:2602.11000*, 2026.

[18] H. Bai, L. Kong, X. Chen, et al. DICE: Diffusion large language models excel at generating CUDA kernels. *arXiv preprint arXiv:2602.11715*, 2026.

[19] T. Chen, Z. Ye, B. Xu, et al. AVO: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026.

[20] S. Cao, Z. Mao, J. E. Gonzalez, and I. Stoica. K-Search: LLM kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026.

[21] A. Bhattacharjee, H. Ping, S. V. Le, et al. OptiML: An end-to-end framework for program synthesis and CUDA kernel optimization. *arXiv preprint arXiv:2602.12305*, 2026.

[22] K. S. Dong, S. Modi, D. Nikiforov, et al. KernelBlaster: Continual cross-task CUDA optimization via memory-augmented in-context reinforcement learning. *arXiv preprint arXiv:2602.14293*, 2026.

[23] X. Cao, J. Zhai, P. Li, et al. AscendKernelGen: A systematic study of LLM-based kernel generation for neural processing units. *arXiv preprint arXiv:2601.07160*, 2026.

[24] Z. Wen, S. Shao, Z. Li, et al. AscendCraft: Automatic Ascend NPU kernel generation via DSL-guided transcompilation. *arXiv preprint arXiv:2601.22760*, 2026.